

COMP 3300 – Fall 2025
Assignment 2, 10 pts.
Due: Sunday, August 31st at
11:55pm

Objectives

- Create a C# application that has separation of concerns with model classes and a driver class.
- Use C# properties and lists.
- Compute basic stats of data contained in a list, e.g., min, max, average.

Notes

- Please read through this entire document before you begin, so you know all the requirements for the program at the outset.
- **Any program that does not compile will receive a zero (0). Partial credit is not possible for any program that does not compile.**
- **No questions will be answered regarding the assignment via email, f2f, phone, virtual meeting, or otherwise after 5p on the due date.**

Background

In this assignment we are going to simulate some possible functionality for a video game. In this assignment we are going to develop some code that implements a few pieces for the game. To start we are going to develop a solid structure for how we are going to start the application, the *Driver* class. This class will be responsible for “kicking off” our simulation.

Next up we will implement both a *Level* class as well as an *Attempt* class. The *Level* will represent a map (think something akin to a dungeon or stage) and the *Attempt* will represent metrics for an attempt at a given *Level*. You will develop each class to accomplish the goals set for this deliverable.

Getting Started

1. Before you start make sure you watch the C# `properties` and `lists` videos and step through the coding activity detailed in it. This application will have some similar functionality present (functionally similar) within the `Attempt` class and this application will also have a `Level` class.
2. Start Visual Studio 2022 and create a new C# `Console Application` that is a .NET application, i.e., it can run on Windows, Linux, and MacOS.
 - a. Name the project `FirstnameLastnameA2`, e.g., `SallieMaeA2`.
 - b. Create the application with a `Main` method. To do this make sure to **check** Do not use top-level statements.
3. Build and run the application to verify it runs. It should display `Hello, World` in the console application window that is created upon running the application.

Requirements

Introduce a *Driver* class:

- 1) The `Main` method should not have any functionality in it, other than starting the “driver” method of the class that the main method is contained within. To do this, do the following:
 - a) Create a folder/namespace inside the project with the name “Driver”.
 - i) Within the folder create a class named “Driver”.
 - b) Add an instance method (non-static method) to the `Driver` class named `Run`.
 - c) The `Run` method should return void and have no parameters.
 - d) In the `Run` method have it output the following:

```
Firstname Lastname Assignment 2
```

e.g., Sallie Mae Assignment 2.
- 2) In the `Main` method, do the following:
 - a) Delete the line that outputs `Hello, World`.
 - b) Create an instance of the new class object that runs our application.
 - i) This MAY require you import the new class namespace to the file.
 - c) Using the class object, call the `Run` method.
4. Build and run the application to verify it runs correctly.

Model namespace and classes

1. Create a `model` folder/namespace.
2. Within the `Model` folder/namespace, create an `Attempt` data class that has the following properties:
 - a. Auto-implemented read/write properties:
 - i. `FirstName` : string
 - ii. `LastName` : string
 - iii. `Score` : int
 - iv. `Time` : decimal
 - v. `RunDate` : DateTime
 - b. Each of these properties should be declared to be nullable.
 - c. Expression-bodied properties:
 - i. `PlayerName` – concatenation of first and last name with a space in between.
 - ii. `IsTimeBelowRecord` – returns true if the *Time* is less than 27 1/2 seconds.
 - It may possible that *Time* could be null, so you need to handle that.

3. Add the following constructors to the *Attempt* class:

c. Default constructor that initializes the properties as follows:

- i. First and last name to empty strings.
- ii. Score and Time to zero.

d. A four-parameter constructor that accepts the first and last name, time, and score as parameters and sets the corresponding properties.

i. *RunDate* should default the current Date/Time.

ii. In addition, it should do the following precondition checks:

- 1. The first and last name parameters should not equal `null`, if so throw an `ArgumentNullException`.
- 2. Score between 0 and 2000 inclusive, otherwise throw an `ArgumentOutOfRangeException`.
- 3. Time – is between 0.0 and 59.99, if not throw an `ArgumentException`.

4. Add a *ToString* method to the *Attempt* class that returns the following string:

FirstName LastName with a score of (Score) and a time of (Time) on (RunDate).

a. Make sure that *RunDate* displays ONLY the date portion of its data.

5. Within the *Model* folder/namespace, create a *Level* class that has the following properties:

a. Auto-implemented properties:

- i. *RunAttempts* : `List<Attempt>` - read only.
- ii. *LevelName* : `string` - read/write.

b. Expression-bodied property:

- i. *RunCount* which returns the number of *RunAttempt* objects in the collection.
- ii. *RecordRunCount* which returns the number of *Attempts* that have a time below the record.

6. Add the following constructors to the *Level* class:

a. Add a default constructor that just instantiates the *RunAttempts* property object and sets the *LevelName* property to an empty string.

7. Add a one-parameter constructor that takes the `LevelName` as a parameter and initialize the `LevelName` property with the parameter value. Initialize the `RunAttempts` property appropriately within the constructor.
8. Add an `AddLevel` method to the `Level` class that accepts an *Attempt* object as a parameter and then adds the attempt to the collection of *RunAttempts*. This method should not return anything.
9. Overload the `AddLevel` method with another one that accepts four parameters, a first and last name, score, and time and then it creates an *Attempt* object based on the data passed in and adds the student to the collection of *RunAttempts*.
10. Add a `ToString` method that returns the following string:

Level: (LevelName) - Total Attempts: (RunCount), Record Runs: (RecordRunCount)

For example:

Level: Desert - Total Attempts: 7 Record Runs: 3

Create a `Level` that can be populated:

1. Add a private `Level` field/data member to the `Program` class.
2. Add a default constructor the `Program` class and instantiate the `Level` data member.

Fill the `Level Attempts`:

Note: For the remaining requirements, I will not provide details as to where specifically the functionality needs to go within the `Program` class. You can introduce methods as appropriate within this class and then call them from the `Run` method.

1. Add functionality to the `Program` class that will prompt the user for the name of the `Level` and the number of attempts to add to it.

Hint: Remember, the `ReadLine` method returns a string, but you can convert a string to an integer.

2. Add functionality that will prompt for the first name, last name, score, and time for EACH of the number of attempts to add.
 - There should be between 3 and 5 attempts added in.

3. After the Attempt information has been read in for the specified number of attempts. Print out the level name, total attempts, and the list of attempt scores as follows:

Level Name – Total Attempts: Count

Attempts:

FirstName LastName (Score) : Time

FirstName LastName (Score) : Time

The IDs and grades must be aligned in a column.

Example

Desert – Total Attempts: 3

Roster:

Sally Mae (798) : 45.9

Roger Wilson (573) : 56.3

Misty Green (1000) : 14

4. Build and run the application to verify it runs correctly.

Additional roster stats

1. In the `Level` class add the following methods:
 - a. A method that finds and returns the minimum score for the attempts in the level.
 - b. A method that finds and returns the maximum score for the attempts in the level.
 - c. A method that finds and returns the average time for the attempts in the level.
 - d. A method that finds and returns the time range for the attempts in the level.

For example, if the highest time was a 59 and the lowest a 45, this method would return 14.

- e. A method that finds and returns the count of the number of scores between two parameter values inclusive. The two parameter values should be the low and high values.

Note: Inclusive means that if it equals the low or high value it should be counted.

2. In the `Program` class, update the output to include the stats after the level data. The average must be print out to two decimal places.

Example

Desert – Total Attempts: 5

Attempts:

```
Sally Mae      (798) : 48.9
Roger Wilson   (1500) : 17
Misty Green    (547) : 36
Seth Brown     (998) : 59.7
Amber Smith    (1750) : 7
```

Stats:

```
Min: 547
Max: 1750
Ave: 33.72
Range: 52.7
Scores between 1500 and 2000, inclusive: 2
```

3. Build and run the application to verify it runs correctly.

Documentation

1. For all the class header and the public methods add in the XML documentation header and complete it.

Grading breakdown

- 10 pts. – Exercise completed successfully with at most only a minor error and follows best programming practices.
- 7 pts. – Exercise is missing some functionality or has some errors in it and mostly follows best programming practices.
- 4 pts. – Exercise is missing most of the major functionality or has major errors or best programming practices not followed.
- 0 pts. – Exercise not submitted, does not compile, or barely started.

Submission

Make sure you clean your solution (**Build → Clean Solution**) and then zip up your project which needs to include the solution file into a ZIP file called *YourNameA2.zip* and submit the assignment by the due date.